03.14.2026
Dias Almat
Shokhabbos Mukhammatov
CSC 665

Homework 2

Problem I

Part 1

b)

How you represent nodes and track visits/values:

The nodes are defined by the MCTSNode class. Each node stores the game state, the parent reference, parent action, pointer to children, untried actions and N and W values. N represents the number of times the node has been visited and W represents the total reward from the simulations that passed through the node.

How you choose which unexpanded action to expand:

Each node contains a list that keeps track of untried actions. During the expansion an untried action is selected randomly and is removed from the list.

What rollout policy you used.

We used a random policy for the rollout phase. Starting from the node, the algorithm repeatedly selects a random action till the terminal state. Once the terminal state is reached, the reward is computed and the value of the node is changed.

Part 2

Budget	Win Rate	Avg Score Diff	Avg ms/move MCTS	Avg ms/move MiniMax
20	88.3%	12.70	0.14	328.85
200	91.7%	16.13	1.17	311.82
2000	93.3%	15.13	11.35	315.41
20000	95.0%	15.70	111.94	311.58

Part 3

1.
    a. The performance increases as the budget increases. This is because with increasing budget the MCTS explores more of the tree, and can make more precise decisions
    b. With the budget increasing, the time spent per move also increases. That is because now the algorithm has to traverse more nodes per each move.

2.
   a. Yes, this outcome is logical for MCTS. It is expected because MCTS goes through multiple simulations depending on the budget, covering more possible options. Also, MCTS updates the values of the nodes based on the outcome of every simulation, giving a more precise answer.
   b. One of the reasons is because the game space is comparatively small. At each turn there are only a few legal actions, so MCTS can repeatedly test many options and identify which moves would lead to a better outcome. Another reason is the implementation choices. In our implementation MCTS is allowed to run many simulations based on the budget, while the minimax follows a fixed search process.
   c. It may be possible to make the two algorithms more equivalent in performance by reducing the MCTS advantage. One of the ways is using a very small budget for MCTS. If we lower the budget, then MCTS will have less simulations to find the better option. Another way could be by changing the rollout policy of MCTS.
3. In part 2, the win rate did not degrade with the budget increasing on the first run. However, during the second run, we noticed that the win rate stopped improving after the budget of 200, staying at 91.7%. One of the reasons why that happens could be because MCTS relies on random rollouts. It can produce noisy estimates, especially if some positions require more careful consideration than random play.

Problem II

Writting part: Part 1(a) – XOR using only $\wedge$, $\vee$, $\neg$

Answer:

$$P \oplus Q \equiv (P \vee Q) \wedge \neg(P \wedge Q)$$

Justification:

So, the conjunct **$(P \vee Q)$** ensures that **at least one** of P or Q is true and the conjunct **$\neg(P \wedge Q)$** ensures **not both** are true simultaneously. That's why? together they enforce that **exactly one** of P or Q is true — which is precisely the definition of $P \oplus Q$.

Truth Table Verification:

P	Q	$P \vee Q$	$\neg(P \wedge Q)$	$(P \vee Q) \wedge \neg(P \wedge Q)$	$P \oplus Q$
F	F	F	T	F	F
F	T	T	T	T	T
T	F	T	T	T	T
T	T	T	F	F	F

Problem III

Part 1

a. $\forall x \exists y(Successor(x,y) \wedge y \neq x \wedge \forall z(Successor(x,z) \rightarrow z=y))$
b. $\forall x((Odd(x) \vee (Even(x)) \wedge \neg(Odd(x) \wedge Even(x)))$
c. $\forall x \forall y((Even(x) \wedge Successor(x,y)) \rightarrow Odd(y))$
d. $\forall x \forall y((Odd(x) \wedge Successor(x,y)) \rightarrow Even(y))$
e. $\forall x \forall y(Successor(x,y) \rightarrow Larger(x,y))$
f. $\forall x \forall y \forall z((Larger(x,y) \wedge Larger(y,z)) \rightarrow Larger(x,z))$
g. Consequence: $\forall x \exists y(Larger(y,x) \wedge Even(y))$